Session 9: Transactions & Error HandlingTrainer Coverage:COMMIT, ROLLBACK, SAVEPOINT.Triggers: use case “Auto update user’s balance after expense entry.”AI Integration: Use ChatGPT to simulate test triggers and rollback scenarios.Pending

Tasks

1) **Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.**

Query:

```sql
-- Create Orders table (run only if it doesn't already exist)
CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100) NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    quantity INT NOT NULL,
    order_date DATE
);

-- Start the transaction
START TRANSACTION;

-- Insert a new order
INSERT INTO Orders (customer_name, product_name, quantity, order_date)
VALUES ('Abhi', 'Laptop', 1, CURDATE());
```

```sql
VALUES ('Abhi', 'Laptop', 1, CURDATE());

-- Save the transaction
COMMIT;

-- Verify that the order was saved
SELECT * FROM Orders;
```

Output:

Extra options

←T→	order_id	customer_name	product_name	quantity	order_date
Edit Copy Delete	1	Abhi	Laptop	1	2026-09-01

2) **Simulate a Zomato-style food order process: insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.**

Query:

Run SQL query/queries on database my database:

```sql
-- Create OrderItems table
CREATE TABLE OrderItems (
    item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

-- Start transaction
START TRANSACTION;

-- Insert first food item
INSERT INTO OrderItems (order_id, item_name, quantity, price)
VALUES (101, 'Veg Pizza', 1, 250.00);
```

```sql
VALUES (101, 'French Fries', 2, 120.00);

-- Check the inserted items before rollback
SELECT * FROM OrderItems;

-- Undo both INSERT operations
ROLLBACK;

-- Verify that the new items are gone
SELECT * FROM OrderItems;
```

Output:

Extra options

←T→	item_id	order_id	item_name	quantity	price
Edit Copy Delete	1	101	Veg Pizza	1	250.00
Edit Copy Delete	2	101	French Fries	2	120.00

Check all *With selected:* Edit Copy Delete Export

3) **In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.**

Query:

```sql
-- Create Cart table
CREATE TABLE Cart (
    cart_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

-- Start transaction
START TRANSACTION;

-- Add the first product
INSERT INTO Cart (user_id, product_name, quantity, price)
VALUES (1, 'Laptop', 1, 55000.00);
```

```sql
-- Mark a savepoint after adding the first product
SAVEPOINT product_added;

-- Add the second product
INSERT INTO Cart (user_id, product_name, quantity, price)
VALUES (1, 'Wireless Mouse', 1, 800.00);

-- Undo only the second product
ROLLBACK TO SAVEPOINT product_added;

-- Save the remaining transaction
COMMIT;

-- Show final contents of Cart
SELECT * FROM Cart;
```

Output:

Extra options

	cart_id	user_id	product_name	quantity	price
Edit Copy Delete	1	1	Laptop	1	55000.00

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table

4) **Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.**

Query:

```sql
-- Create Wallet table
CREATE TABLE Wallet (
    user_id INT PRIMARY KEY,
    user_name VARCHAR(100),
    balance DECIMAL(10,2) NOT NULL
);

-- Create Transactions table
CREATE TABLE Transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Insert sample wallet balance
INSERT INTO Wallet (user_id, user_name, balance)
VALUES (1, 'Abhi', 5000.00);

-- Create trigger
DELIMITER //

CREATE TRIGGER deduct_wallet_balance
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END //
```

Output:

Extra options

←T→				user_id	user_name	balance
☐	Edit	Copy	Delete	1	Abhi	3800.00

5) **Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error.

Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.**

Query:

```sql
CREATE TABLE Wallet (
    user_id INT PRIMARY KEY,
    user_name VARCHAR(100),
    balance DECIMAL(10,2)
);

CREATE TABLE Expenses (
    expense_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    amount DECIMAL(10,2),
    description VARCHAR(100)
);

INSERT INTO Wallet (user_id, user_name, balance)
VALUES (1, 'Abhi', 5000.00);
```

Run SQL query/queries on database my database:

```sql
DELIMITER //

CREATE TRIGGER buggy_expense_trigger
AFTER INSERT ON Expenses
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance + NEW.amount
    WHERE user_id = NEW.user_id;
END //

DELIMITER ;
```

```sql
INSERT INTO Expenses (user_id, amount, description)
VALUES (1, 1200.00, 'Shopping');

SELECT * FROM Wallet;
```

You have a previously saved qu
query to load th

Run SQL query/queries on table my database.Wallet:

```
DROP TRIGGER buggy_expense_trigger;
```

Run SQL query/queries on table my database.Wallet:

```
DELIMITER //

CREATE TRIGGER correct_expense_trigger
AFTER INSERT ON Expenses
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END //

DELIMITER ;
```

Run SQL query/queries on table my databas

```
UPDATE Wallet
SET balance = 5000.00
WHERE user_id = 1;
```

Run SQL query/queries on table my database.Wallet:

```
INSERT INTO Expenses (user_id, amount, description)
VALUES (1, 1200.00, 'Shopping');
```

Output:

Extra options

	user_id	user_name	balance
Edit Copy Delete	1	Abhi	6200.00

Extra options

	user_id	user_name	balance
Edit Copy Delete	1	Abhi	2600.00